Callback Inheritance

If you subclass a controller containing callbacks, the callbacks will be run on the child objects as well as in the parent. But callbacks defined in the children won't run in the parent.

If you don't want a particular callback to run in a child controller, you can override the default processing with the `skip_before_action` and `skip_after_action` declarations. These accept the `:only` and `:except` parameters.

You can use `skip_action` to skip any action callback (before, after, and around). However, it works only for callbacks that were specified as the (symbol) name of a method.

We made use of `skip_before_action` in *Iteration I3: Limiting Access*.

What We Just Did

We learned how Action Dispatch and Action Controller cooperate to enable our server to respond to requests. The importance of this can't be emphasized enough. In nearly every application, this is the primary place where the creativity of your application is expressed. While Active Record and Action View are hardly passive, our routes and our controllers are where the action is.

We started this chapter by covering the concept of REST, which was the inspiration for the way in which Rails approaches the routing of requests. We saw how this provided seven basic actions as a starting point and how to add more actions. We also saw how to select a data representation (for example, JSON or XML). And we covered how to test routes.

We then covered the environment that Action Controller provides for your actions as well as the methods it provides for rendering and redirecting. Finally, we covered sessions, flash, and callbacks, each of which is available for use in your application's controllers.

Along the way, we showed how these concepts were used in the Depot application. Now that you've seen each in use and have been exposed to the